

RAI-818: INTELLIGENT **MOBILE ROBOTICS**

Bug Algorithms – Part II

- **Tangent Bug**

Text : Principles of Robot Motion – Theory, Algorithms and Implementations
H. Choset, K.M. Lynch, S. Hutchinson, G. Kantor, W. Burgard, et al.

INSTRUCTOR:

DR. KUNWAR FARAZ AHMED KHAN

Bug Algorithms



- Special Bugs?
- General Concepts
- Bug 0
- Bug 1
- Bug 2
- **TangentBug**
 - **Realistic Bug**

A More Realistic Bug

- Bug 1 and bug 2 assume knowledge of goal position and uses contact-based wall following
- In reality, we typically use some sort of range sensing device that lets us look ahead
 - but a sensor has finite resolution and is noisy
- Let us assume we have a range sensor



Defining futures

Bug Algorithms



- Special Bugs?
- General Concepts
- Bug 0
- Bug 1
- Bug 2
- **TangentBug**
 - **Realistic Bug**

A More Realistic Bug

- We model the range sensor with a *raw distance function*

$$\rho : \mathbb{R}^2 \times S^1 \rightarrow \mathbb{R}$$

- The *raw distance function* is given by

$$\rho(x, \theta) = \min_{\lambda \in [0, \infty]} d(x, x + \lambda [\cos \theta, \sin \theta]^T),$$

$$\text{such that } x + \lambda [\cos \theta, \sin \theta]^T \in \bigcup W O_i$$

- Since θ is infinite there can be an infinite resolution
- Approximate with a finite number of range sensors along the circumference



Defining futures

Bug Algorithms



- Special Bugs?
- General Concepts
- Bug 0
- Bug 1
- Bug 2
- TangentBug
- Realistic Bug

A More Realistic Bug

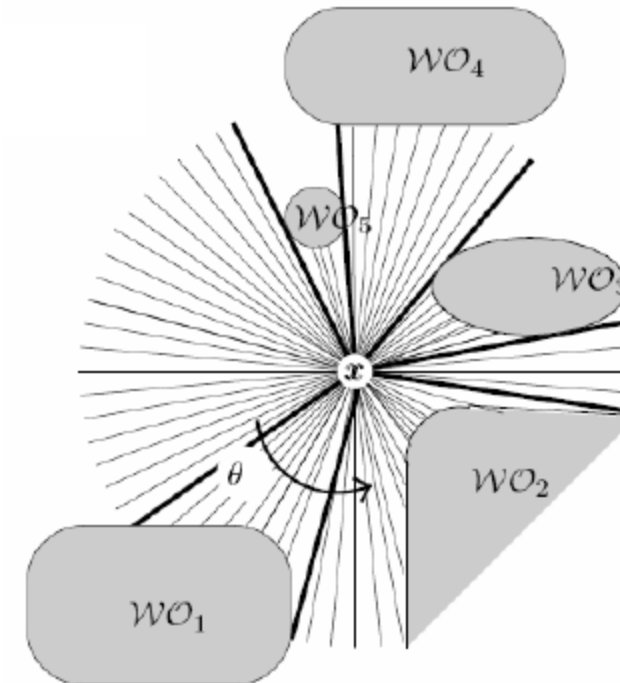
- Since real sensors have a limited range R we define a *saturated raw distance function*

$$\rho_R : \mathbb{R}^2 \times S^1 \rightarrow \mathbb{R}$$

- The *saturated raw distance function* is given by

$$\rho_R(x, \theta) = \begin{cases} \rho(x, \theta) & \text{if } \rho_R(x, \theta) < R \\ \infty & \text{otherwise} \end{cases}$$

The thin lines are values of $\rho_R(x, \theta)$ and the thick lines indicate discontinuities



Defining futures



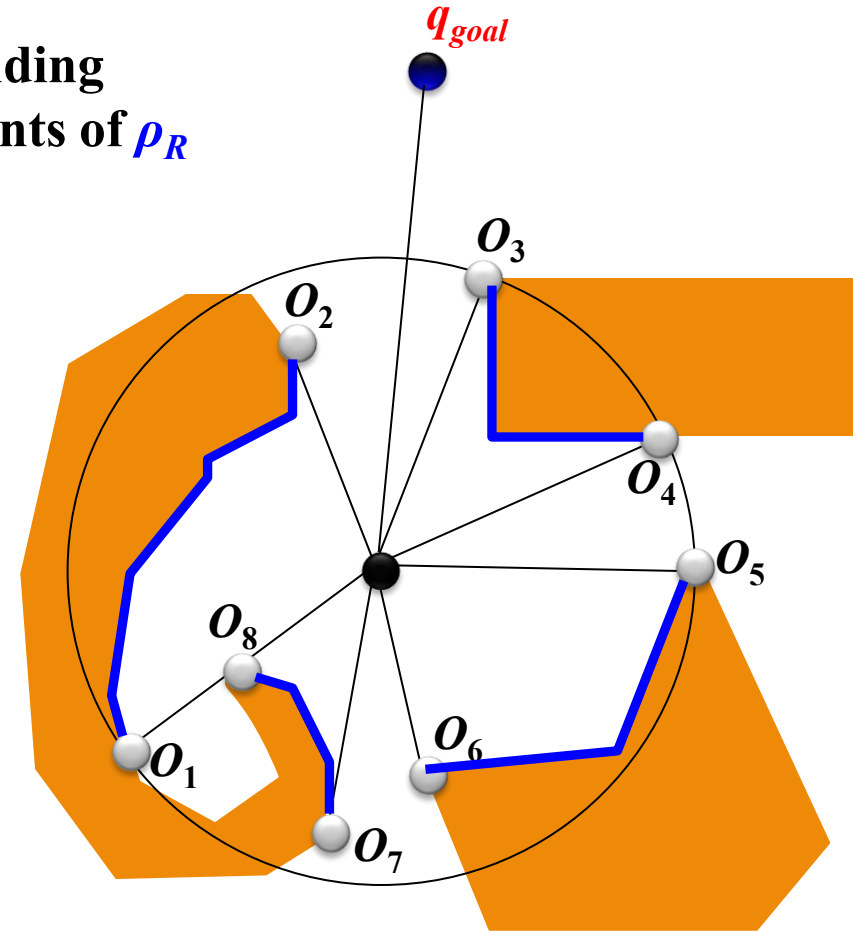
Bug Algorithms

- Special Bugs?
- General Concepts
- Bug 0
- Bug 1
- Bug 2
- **TangentBug**
 - Realistic Bug
 - **Continuity**

Intervals of Continuity

Tangent Bug relies on finding endpoints of finite segments of ρ_R

The points of discontinuities of $\rho_R(x, \theta)$ correspond to points O_i on the obstacles. The thick solid curves represents intervals of continuity.



The robot to the best of its sensing range believes that there is a straight –line path to the goal



Bug Algorithms

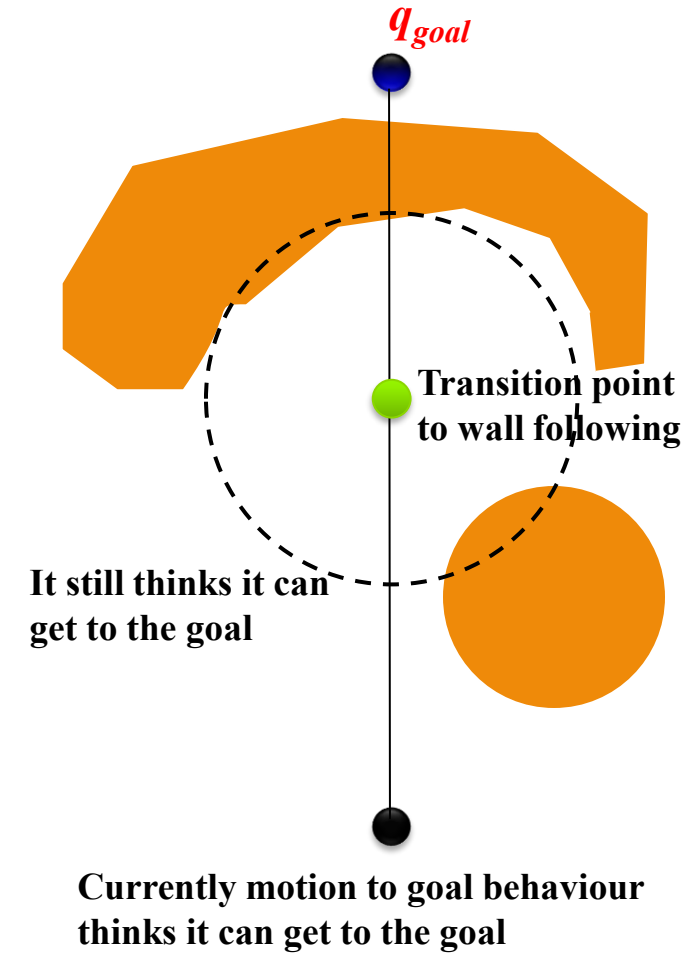


- Special Bugs?
- General Concepts
- Bug 0
- Bug 1
- Bug 2
- **TangentBug**
 - Realistic Bug
 - Continuity
 - **Tangent Behavior**

Tangent Bug Behaviour

- Just like Bug 1 and Bug 2 the Tangent Bug iterates between
 - Motion to goal
 - Wall following
- Motion to goal has two portions
 - Move toward goal until an obstacle is sensed directly between the robot and the goal form the O_i endpoints
 - Move in a direction that minimizes a heuristic

$$d(x, O_i) + d(O_i, q_{goal})$$



Defining futures

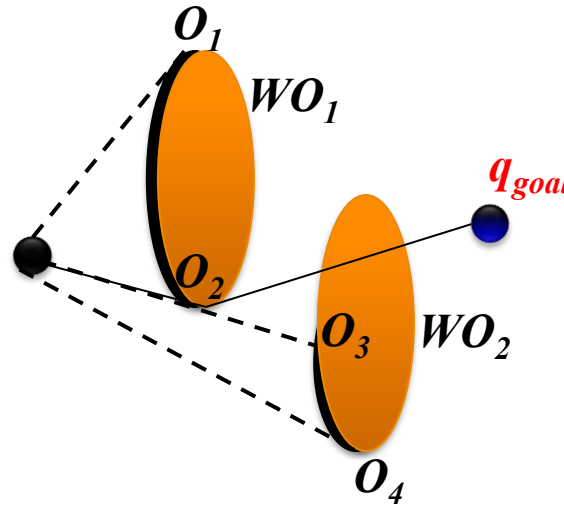


Bug Algorithms

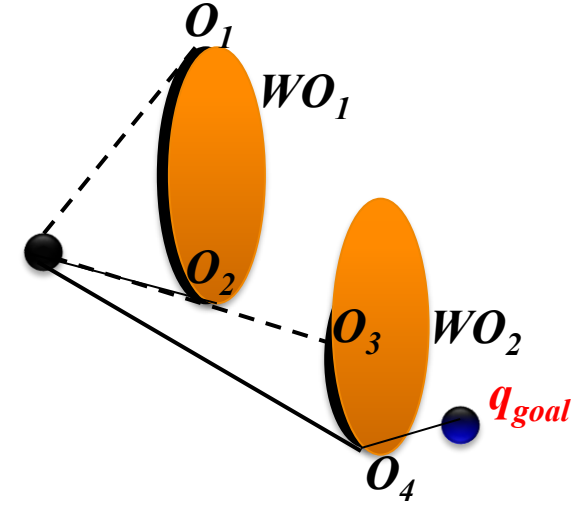
- Special Bugs?
- General Concepts
- Bug 0
- Bug 1
- Bug 2
- **TangentBug**
 - Realistic Bug
 - Continuity
 - **Tangent Behavior**

Minimize Heuristic Example

at x , robot know only what it sees and where the goal is



The planner selects O_2 as a sub-goal for the robot



The planner selects O_4 as a sub-goal for the robot

Choose the point O_i that minimizes $d(x, O_i) + d(O_i, q_{goal})$



Defining futures

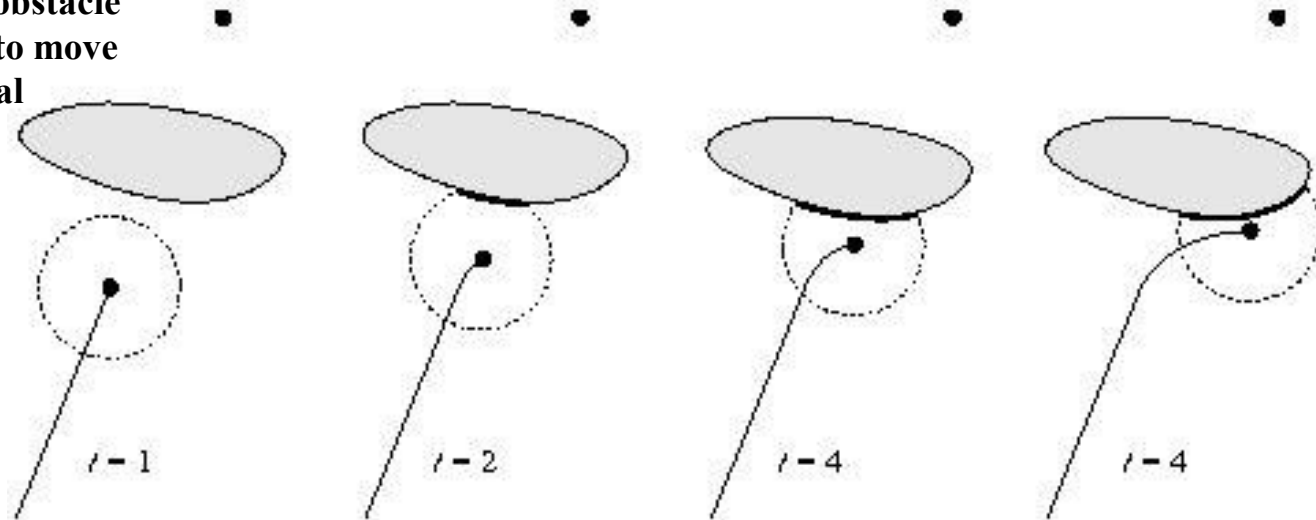


Bug Algorithms

- Special Bugs?
- General Concepts
- Bug 0
- Bug 1
- Bug 2
- **TangentBug**
 - Realistic Bug
 - Continuity
 - **Tangent Behavior**

Motion to Goal Example

Robot has not
sensed the obstacle
-continues to move
towards goal



Robot initially senses the
obstacle-uses heuristics
to decide direction

Robot senses more of the
obstacle-uses heuristics
to decide direction

Choose the point O_i that minimizes $d(x, O_i) + d(O_i, q_{goal})$



Defining futures

Bug Algorithms



- Special Bugs?
- General Concepts
- Bug 0
- Bug 1
- Bug 2
- **TangentBug**
 - Realistic Bug
 - Continuity
 - **Tangent Behavior**

Tangent Bug Behavior

- Choose the point O_i that minimizes $d(x, O_i) + d(O_i, q_{goal})$
- **Problem:** what if this distance starts to go up?
- **Answer:** start to act like a BUG and follow boundary

Maintain :

$d_{followed}$ and $d_{reached}$

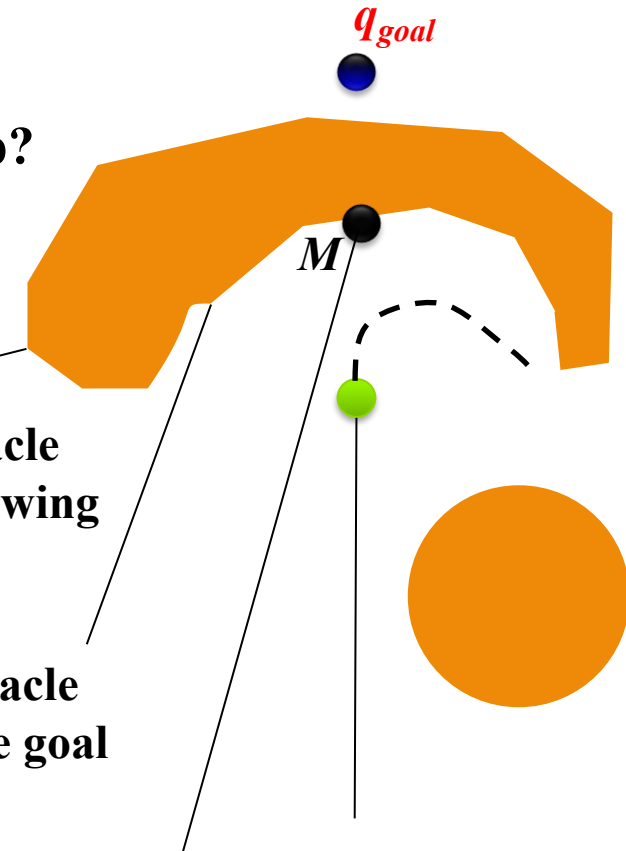
At start:

$d_{followed}$ is same as $d_{reached}$

followed obstacle the obstacle that we are currently following

blocking obstacle the obstacle that is between us and the goal

M is the pt on the sensed obstacle which has the shortest distance to the goal



Defining futures

Bug Algorithms



- Special Bugs?
- General Concepts
- Bug 0
- Bug 1
- Bug 2
- **TangentBug**
 - Realistic Bug
 - Continuity
 - **Tangent Behavior**

Tangent Bug Behavior

- $d_{followed}$ is the shortest distance between the sensed boundary and the goal
- d_{reach} is the shortest distance between *blocking obstacle* and *goal*
$$\Lambda = \{y \in \partial WO_b : \lambda x + (1 - \lambda)y \in Q_{free} \ \forall \lambda \in [0,1]\}$$
$$d_{reach} = \min_{c \in \Lambda} d(q_{goal}, c)$$

(or my distance to goal if no blocking obstacle visible)
- Terminate boundary following behaviour when
 $d_{reach} < d_{followed}$



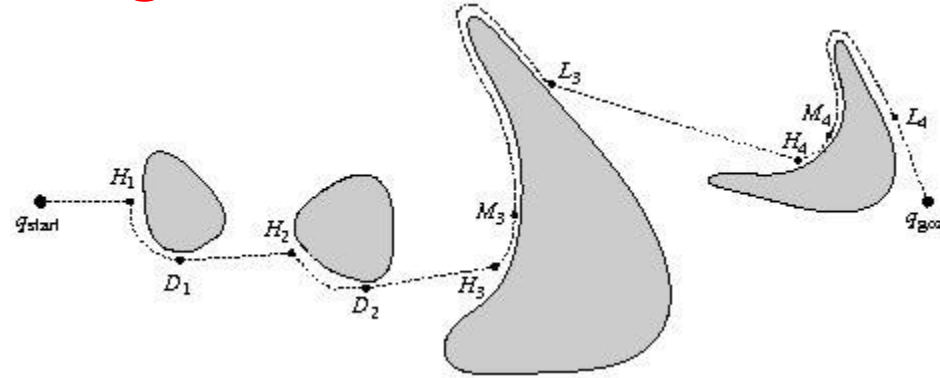
Defining futures

Bug Algorithms



- Special Bugs?
- General Concepts
- Bug 0
- Bug 1
- Bug 2
- **TangentBug**
 - Realistic Bug
 - Continuity
 - **Tangent Behavior**

Tangent Bug Behavior



- Robot moves toward goal until it hits **obstacle 1** at H_1
- Pretend there is an infinitely small sensor range and the O_i which minimizes the heuristic is to the right
- Keep following obstacle until robot can go toward obstacle again
- Same situation with second obstacle
- At third obstacle, the robot turned left until it could not increase heuristic
- $d_{followed}$ is distance between $M3$ and goal, d_{reach} is distance between robot and goal because sensing distance is zero



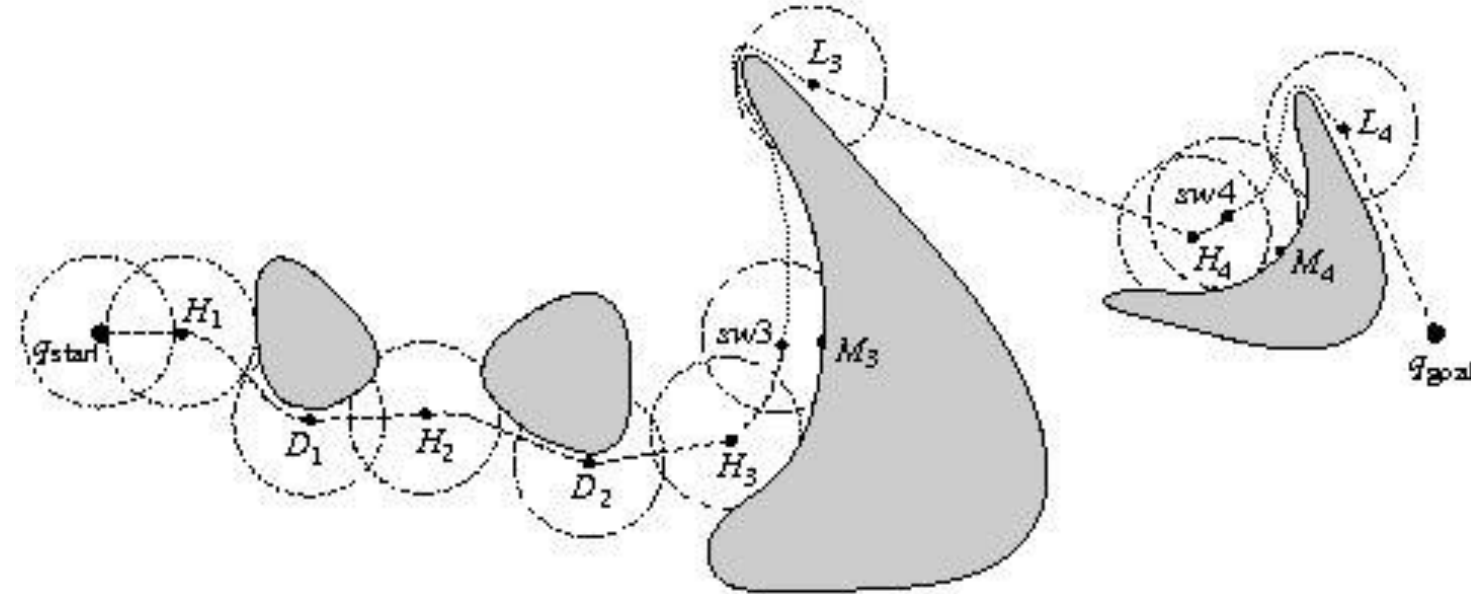
Defining futures

Bug Algorithms



- Special Bugs?
- General Concepts
- Bug 0
- Bug 1
- Bug 2
- **TangentBug**
 - Realistic Bug
 - Continuity
 - **Tangent Behavior**

Tangent Bug Behavior



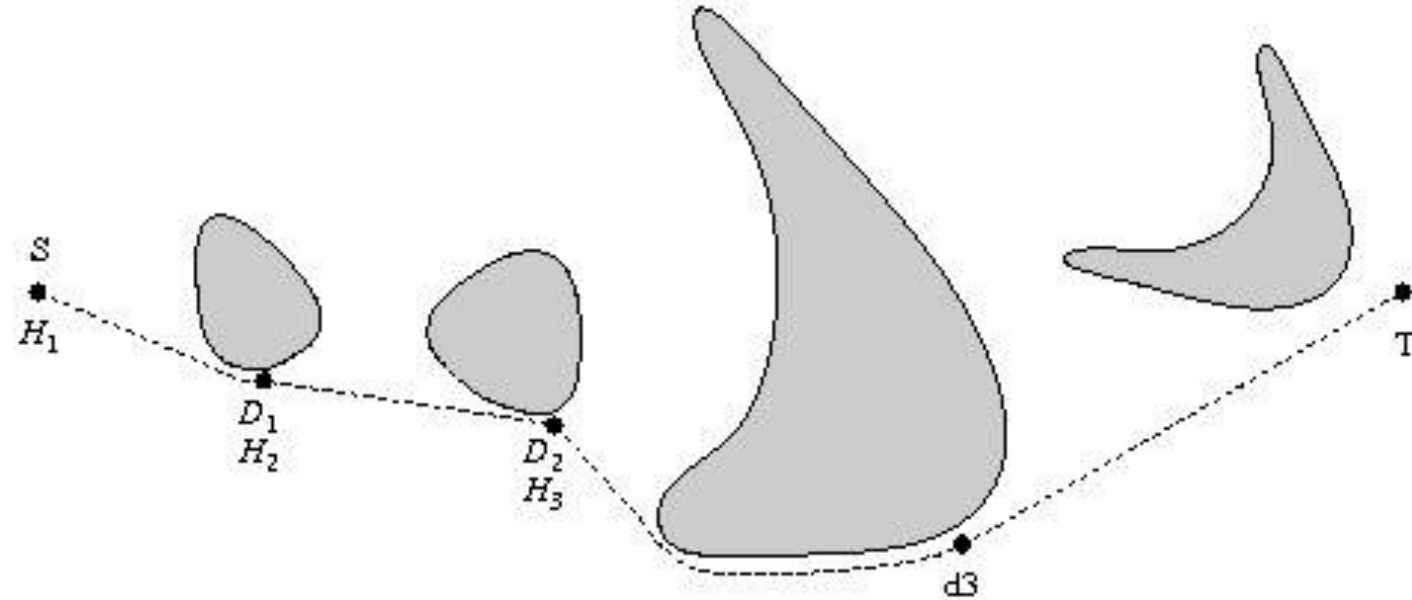
Defining futures

Bug Algorithms



- Special Bugs?
- General Concepts
- Bug 0
- Bug 1
- Bug 2
- **TangentBug**
 - Realistic Bug
 - Continuity
 - **Tangent Behavior**

Tangent Bug Behavior



Defining futures

Bug Algorithms



- Special Bugs?
- General Concepts
- Bug 0
- Bug 1
- Bug 2
- **TangentBug**
 - Realistic Bug
 - Continuity
 - **Tangent Behavior**

Tangent Bug Behavior - Summarized

- A motion-to-goal behaviour as long as way is clear or there is a visible obstacle boundary pt that decreases heuristic distance
- A boundary following behaviour invoked when heuristic distance increases.
- A value $d_{followed}$ which is the shortest distance between the sensed boundary and the goal
- A value d_{reach} which is the shortest distance between **blocking** obstacle and goal (or my distance to goal if no blocking obstacle visible)
- Terminate boundary following behaviour when $d_{reach} < d_{followed}$



Defining futures



Bug Algorithms

- Special Bugs?
- General Concepts
- Bug 0
- Bug 1
- Bug 2
- **TangentBug**
 - Realistic Bug
 - Continuity
 - **Tangent Behavior**

Tangent Bug Formally

Input

*A point robot
with a tactile
sensor*

Output

*A path to q_{goal}
or conclusion
that no such
path exists*

1: repeat

- Compute continuous range segments in view
- Move toward $n \in \{T, O_i\}$ that minimizes
$$h(x, n) = d(x, n) + d(n, q_{goal})$$

until

- goal is encountered, or
- the value of $h(x, n)$ begins to increase

2: follow boundary continuing in same direction as before repeating

- update $\{O_i\}$, d_{reach} and $d_{followed}$

until

- goal is reached
- a complete cycle is performed (goal is unreachable)
- $d_{reach} < d_{followed}$

Note the same general proof reasoning as before applies, although the definition of hit and leave points is a little trickier



Defining futures

Bug Algorithms



- Special Bugs?
- General Concepts
- Bug 0
- Bug 1
- Bug 2
- TangentBug
- **Summary of Bugs**

Bugs Summarized

- **Bug 1: safe and reliable**
- **Bug 2: better in some cases; worse in others**
- **Should understand the basic completeness proof**
- **Two behaviours: motion-to-goal, boundary following**
- **Tangent Bug: supports range sensing**



Defining futures

Bug Algorithms



- Special Bugs?
- General Concepts
- Bug 0
- Bug 1
- Bug 2
- TangentBug
- Summary of Bugs
- Implementation

Implementation

- Bug Algorithms have two behaviours
 - *drive towards a point*
 - *follow an obstacle*
- Drive towards a point is simply a form of gradient descent
- Second behaviour : boundary following presents a challenge since obstacle boundary is not known *a priori*
- Therefore, robot planner must rely on sensor information to determine path
- Full path will not be determined by one reading so the planner has to be incremental

$$d(a, b) = ((a_x - b_x)^2 + (a_y - b_y)^2)^{1/2}$$

Gradient descent of $d(a, b)$, is simply to decrease distance to the goal

robot



goal



- *Challenge*
 - What info is required?
 - How does the robot infer this information from its sensor data?
 - How does this robot use this information to determine the path?



Defining futures

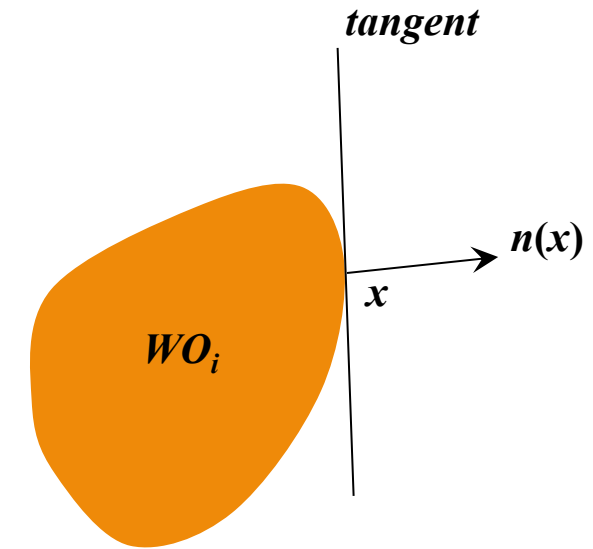
Bug Algorithms



- Special Bugs?
- General Concepts
- Bug 0
- Bug 1
- Bug 2
- TangentBug
- Summary of Bugs
- **Implementation**
 - What Info?

Implementation – What Information?

- For flat obstacles we could just follow a direction parallel to its surface normal $n(x)$
- For obstacles with non-zero curvature robot can follow a path that is consistently orthogonal to the surface normal $n(x)^\perp$
- Assume a locally flat obstacle
 - determine surface normal
 - move orthogonal to surface normal $[\Delta d]$
 - repeat
- The flat line is the tangent to the surface and is a linear approximation of the curve at the point of intersection



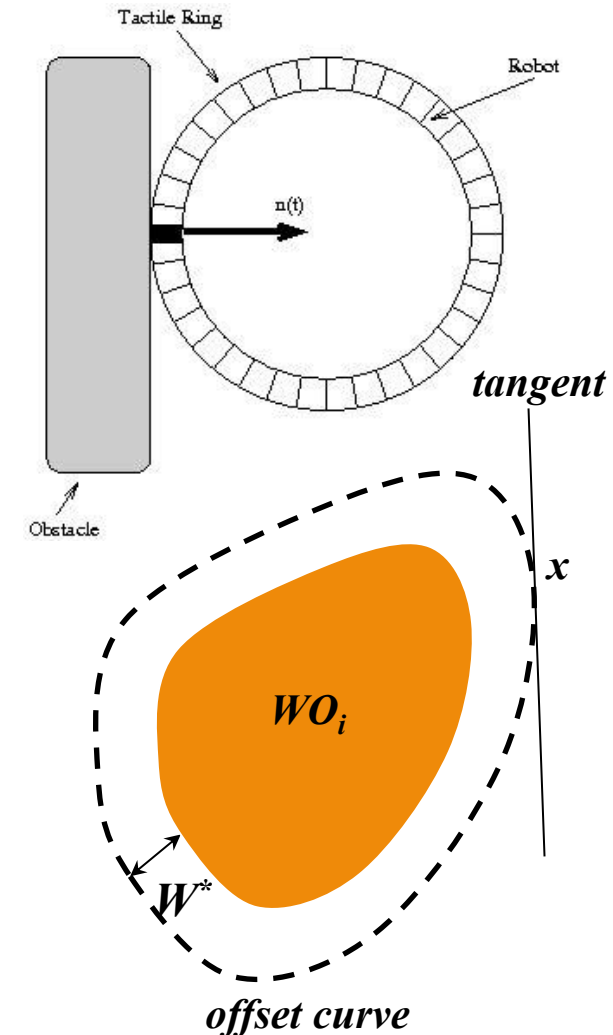
Bug Algorithms



- Special Bugs?
- General Concepts
- Bug 0
- Bug 1
- Bug 2
- TangentBug
- Summary of Bugs
- **Implementation**
 - What Info
 - **How to Infer Info**

Implementation – How to Infer Sensor Information?

- Next step is to infer the tangent from the sensor data
- Consider a circular robot with a fine array of tactile sensors distributed radially across its circumference
- Direction from the contacted sensor to the robot center is $n(x)$
- Using a tactile sensor endangers the robot and the obstacle so robot should follow a path at a safe distance $W^* \in \mathcal{R}$ from the nearest obstacle which is the *offset curve*



Bug Algorithms



- Special Bugs?
- General Concepts
- Bug 0
- Bug 1
- Bug 2
- TangentBug
- Summary of Bugs
- **Implementation**
 - What Info
 - How to Infer Info

Implementation – How to Infer Sensor Information?

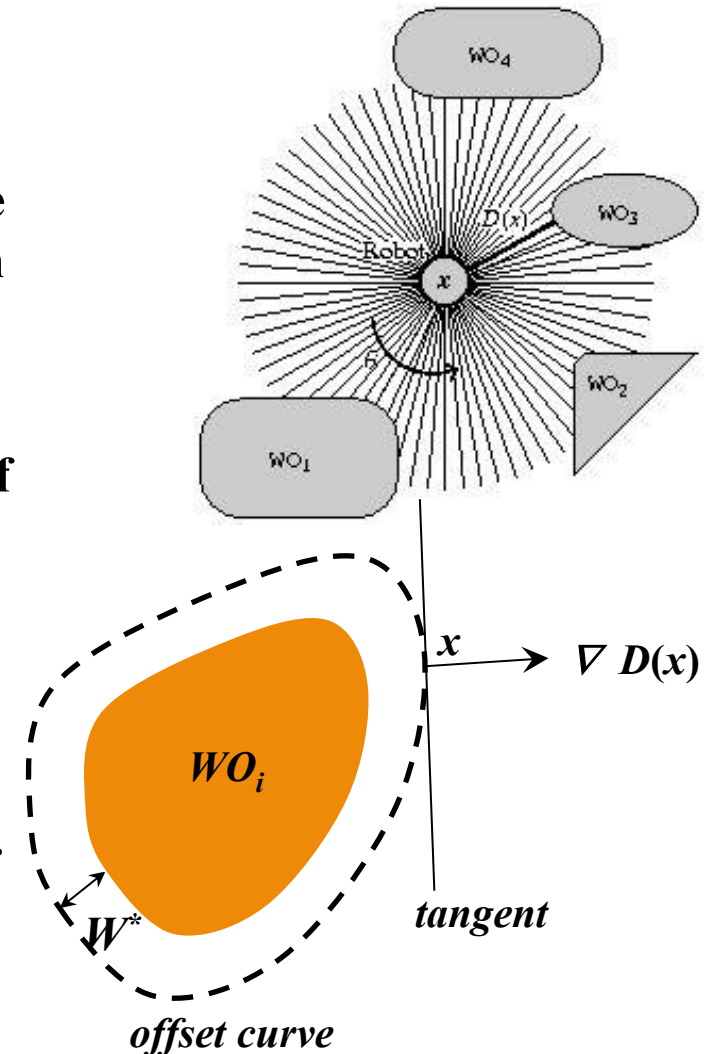
- Let $D(x)$ be distance from x to the closest O_i
$$D(x) = \min_{c \in \cup_i WO_i} d(x, c)$$
- To measure this distance by a robot we use the raw dist fn but now we look for the global min

$$D(x) = \min_s \rho(x, s)$$

- We use gradient of the distance
 - vector that maximally increases the value of a function

$$\Delta D(x) = \left[\frac{\partial D(x)}{\partial x_1} \quad \frac{\partial D(x)}{\partial x_2} \right]^T$$

- Unit direction of smallest value of raw dist fn
- Since the raw dist fn approximates a sensor with individual sensing elements therefore D can be implemented using realistic sensors



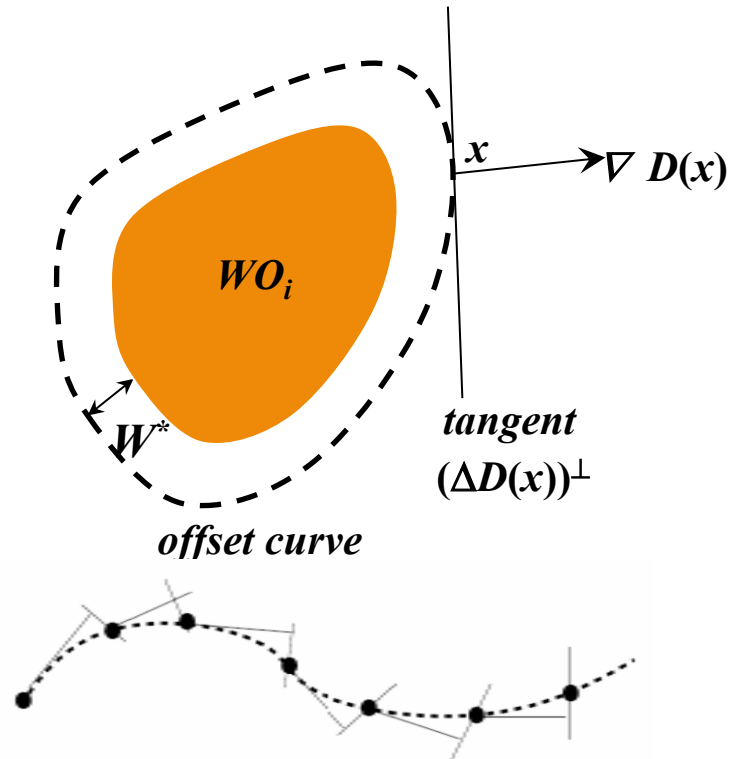
Bug Algorithms



- Special Bugs?
- General Concepts
- Bug 0
- Bug 1
- Bug 2
- TangentBug
- Summary of Bugs
- **Implementation**
 - What Info
 - How to Infer Info
 - **How to Process Info**

Implementation - How to Process Sensor Information?

- *Offset curve* can be defined as a set of points where $G(x) = D(x) - W^*$ maps to zero
- Tangent to the *offset curve* is $(\Delta D(x))^\perp$ which locally maintains distance to the obstacle
- Using distance information robot can determine $(\Delta D(x))^\perp$
- If obstacle is flat *offset curve* is flat
- For an obstacle with curvature we can assume local flatness
- Robot can then move along tangent for a short distance (since the obstacle has a curvature the robot ceases to be on the offset curve)
- To reassess the curve the robot moves either towards or away from the obstacle on $\nabla D(x)$ until it reaches the safety distance W^*



Numerical procedure of *prediction and correction* Robot uses the tangent to predict the shape of the *offset curve* and invokes a correction when the tangent approx becomes invalid



Defining futures

Bug Algorithms



- Special Bugs?
- General Concepts
- Bug 0
- Bug 1
- Bug 2
- TangentBug
- Summary of Bugs
- **Implementation**
 - What Info
 - How to Infer Info
 - How to Process Info
 - **Complete Implementation**

Complete Implementation – Tangent Bug

- **Basic problem:** compute tangent to curve forming boundary of obstacle at any point, and drive the robot in that direction
- Let $D(x) = \min_c d(x, c) \quad c \in U_i \setminus W O_i$
- Let $G(x) = D(x) - W^* \leftarrow$ some safe following distance
- Note that $\nabla G(x)$ points radially away from the object
- Define $T(x) = (\nabla G(x))$ *the tangent direction*
 - in a real sensor this is just the tangent to the array element with lowest reading
- Now we can just move in the direction $T(x)$ or invoke a predictor corrector algorithm



Defining futures